WEEK 10 - PYTHON PROGRAMMING

Your Project Has a History

One fifth of the project grade is a story your commits tell.

ACT 1 · COMMIT	ACT 2 · BRANCH	ACT 3 · PULL REQUEST	LAB
save	try safely	review	ship yours

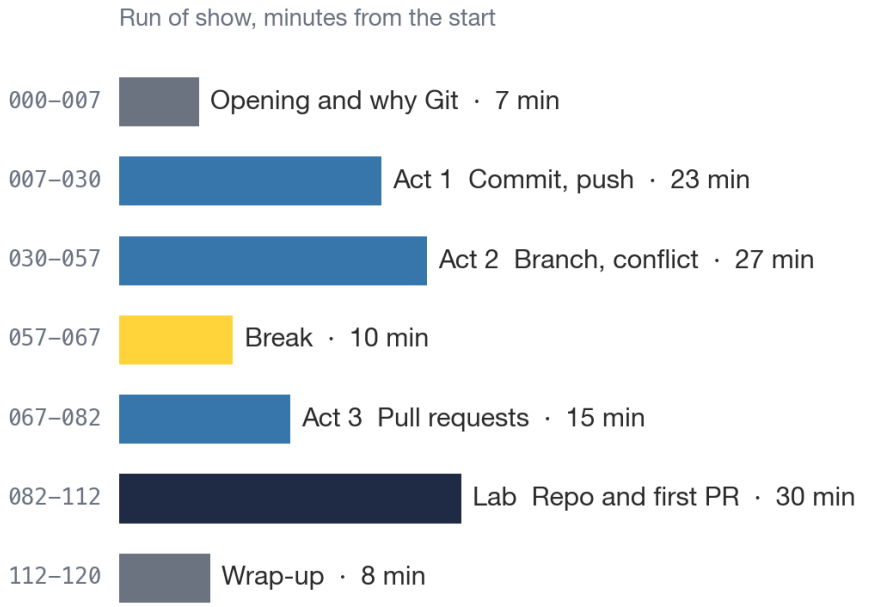
120 minutes, including a break

Instructor: MinGi KYUNG

The Plan: Commit, Branch, Pull Request

Last session you drew the program. Today you give it a home and a history.

- Commit: save points with a message, and a copy on GitHub.
- Branch: try new work without breaking main.
- Pull request: review your own work before it lands.
- Lab: your repository, your first feature branch, your first merged PR.



Why Git When You Work Alone?

Nobody else edits your code. Git still earns its place.

- An undo button for the whole project.
- Proof of process: Git workflow is part of a 20% component.
- A main branch that always runs, ready for the demo.
- Your work on any computer.

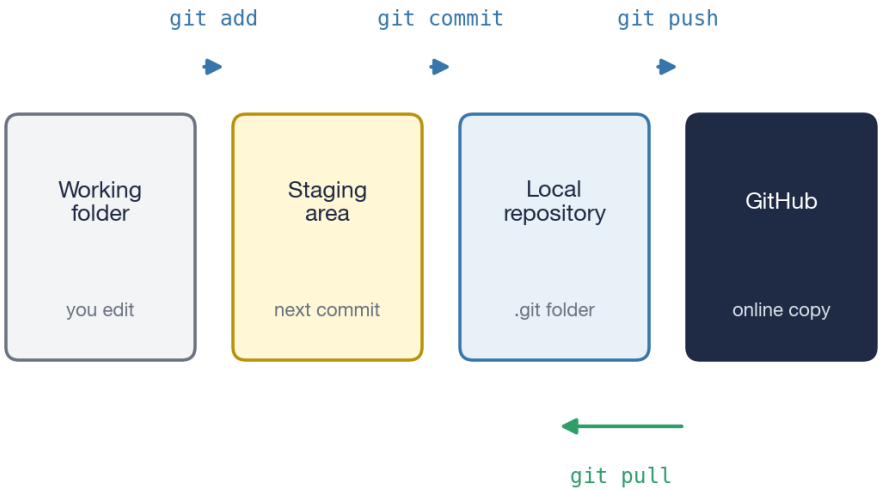
> **TRAP** One commit called "final" in week 15 proves nothing.



Four Places Your Code Lives

Every Git command moves changes between these four places.

- Working folder: the files you edit in VS Code.
- Staging area: the changes you picked for the next commit.
- Local repository: every commit, inside the hidden .git folder.
- GitHub: a copy of the repository online.



git status shows the first three

ACT 1 · COMMIT

Commit: Save Points with a Message

Week 8 in ten minutes, then the habits that earn points.

- init, status, add, commit, log, diff, push.
- Plus the file that keeps junk out: .gitignore.

Once per Computer: Who Are You?

ACT 1 · COMMIT

Git signs every commit with a name and an email. Set them once per laptop.

- Use the email of your GitHub account.
- Name the first branch main in every new project.
- Check the result with git config user.name.

SHELL

```
git config --global user.name "Jiwoo Park"
git config --global user.email "jiwoo@example.com"
git config --global init.defaultBranch main
git config user.name
```

OUTPUT

```
Jiwoo Park
```

Start the Repository

ACT 1 · COMMIT

One git init per project, run inside the project folder.

- git init creates the hidden .git folder. Nothing is saved yet.
- git status tells you what Git sees. Run it constantly.
- The folder also holds __pycache__. Look at the list.

PREDICT __pycache__ is in the folder. Why is it missing from the list?

SHELL

```
mkdir study-logger
cd study-logger
git init
git status
```

OUTPUT

```
Initialized empty Git repository in
  .../study-logger/.git/
On branch main

No commits yet

Untracked files:
        .gitignore
        README.md
        main.py
```

.gitignore: What Never Goes In

A text file of patterns. Matching files are invisible to Git.

- Generated files: __pycache__, .pyc. Python rebuilds them.
- Environments: .venv. Each computer makes its own.
- System junk: .DS_Store, Thumbs.db.
- Secrets: .env, keys, passwords. Never, ever.

TRAP A pushed password is public even after you delete it. Change the password.

OUTPUT

```
# Python
__pycache__/
*.pyc
.venv/

# Editors and operating systems
.vscode/
.DS_Store

# Secrets: never commit these
.env
*.key
```

add, Then commit

ACT 1 · COMMIT

add picks the changes. commit saves them with a message.

- git add . stages everything that is not ignored.
- git commit -m saves one snapshot with a sentence.
- After the commit, the working tree is clean.

SHELL

```
git add .
git commit -m "Add project skeleton"
git status
```

OUTPUT

```
[main (root-commit) c4f930b] Add project skeleton
 3 files changed, 5 insertions(+)
 create mode 100644 .gitignore
 create mode 100644 README.md
 create mode 100644 main.py
On branch main
nothing to commit, working tree clean
```

A Commit Message Is a Sentence

The grader reads your log. Make it read like a diary of decisions.

- Start with a verb in the imperative: Add, Fix, Reject, Rename.
- Say what changed, in about 50 characters.
- One logical change per commit.
- Commit when a small step works, and before you try something risky.

OUTPUT

```
messages that prove nothing
update
asdf
final final2
messages that tell a story
Add load_records stub
Reject negative minutes in parse_minutes
Show weekly totals sorted by minutes
```

See What Changed: diff and log

ACT 1 · COMMIT

Before you commit, look. After you commit, check.

- git diff shows unstaged changes, line by line.
- git commit -am stages tracked files and commits in one move.
- git log --oneline shows one line per commit, newest first.

> **TRAP** -a skips new files. A brand-new storage.py still needs git add.

SHELL

```
git diff
git commit -am "Print the week in the banner"
git log --oneline
```

OUTPUT

```
@@ -1 +1,2 @@
 print("Study logger")
+print("Week 10 skeleton")
[main 71b8a6e] Print the week in the banner
 1 file changed, 1 insertion(+)
71b8a6e Print the week in the banner
c4f930b Add project skeleton
```

Put It on GitHub

ACT 1 · COMMIT

Create the repository online, connect it once, then push.

- On github.com: New repository. Leave it empty: no README, no .gitignore.
- Public or private: private is fine. Then add the instructor as a collaborator.
- push -u the first time. After that, plain git push.

SHELL

```
git remote add origin \
  https://github.com/jiwoo-park/study-logger.git
git push -u origin main
```

OUTPUT

```
To https://github.com/jiwoo-park/study-logger.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

Two Push Errors You Will Meet

Both are normal on the first day. Both have a short fix.

- Rejected: you ticked "Add a README" on GitHub. Run git pull --rebase origin main, then push again.
- Authentication failed: GitHub refuses account passwords on the command line.
- Sign in through VS Code (Source Control, Publish) or gh auth login.

OUTPUT

```
! [rejected]        main -> main (fetch first)
error: failed to push some refs to 'github.com/...'
GitHub has a commit your laptop does not

remote: Support for password authentication
        was removed on August 13, 2021.
fatal: Authentication failed
use VS Code sign-in or gh auth login
```

ACT 2 · BRANCH

Branch: A Safe Place to Try

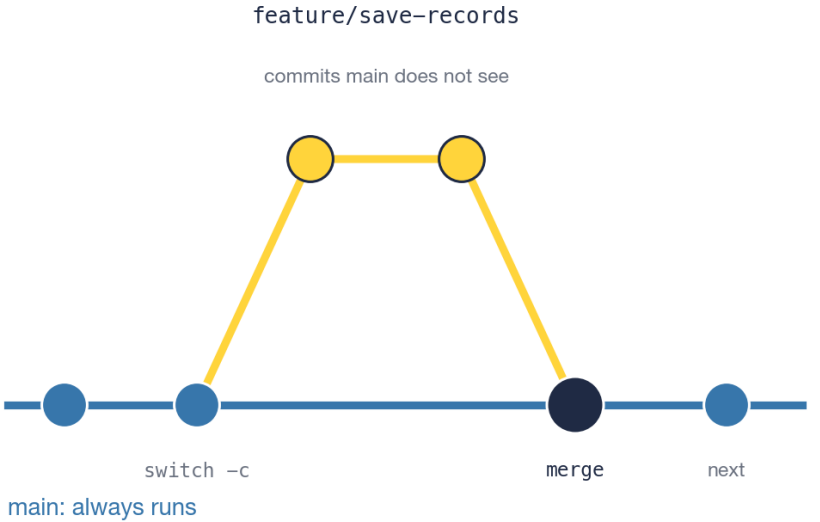
main always runs. New work happens next door.

- One branch per feature, merged when it works.
- Then conflicts: what they look like and how to end them.

Why Branch When You Are Alone?

A branch is a second line of commits that main does not see until you merge.

- main is demo-ready at every moment.
- Each Must feature gets its own short-lived branch.
- A failed experiment is one deleted branch, not a broken project.
- The graph shows the grader exactly how each feature was built.



Create a Branch and Work on It

ACT 2 · BRANCH

switch -c creates the branch and moves you onto it.

- Commits now land on feature/save-records, not main.
- The new file storage.py exists on this branch.

SHELL

```
git switch -c feature/save-records
git add storage.py
git commit -m "Add load_records stub"
ls
```

OUTPUT

```
Switched to a new branch 'feature/save-records'
[feature/save-records d05376a] Add load_records stub
 1 file changed, 2 insertions(+)
 create mode 100644 storage.py
README.md  __pycache__  main.py  storage.py
```

Where Did storage.py Go?

Switch back to main and list the folder again.

- git switch changes the files in your folder to match the branch.
- storage.py is safe. It lives in a commit on the feature branch.
- main has not seen it yet. That is the point.

PREDICT What do you expect ls to show after git switch feature/save-records?

SHELL

```
git switch main
ls
```

OUTPUT

```
Switched to branch 'main'
README.md  __pycache__  main.py
```

Merge: Bring the Work Home

When the feature works, merge it into main from main.

- Fast-forward: main had no new commits, so it just moves up.
- Delete the merged branch. Its commits stay in main.
- log --graph draws the history.

SHELL

```
git merge feature/save-records
git branch -d feature/save-records
git log --oneline --graph
```

OUTPUT

```
Updating 71b8a6e..d05376a
Fast-forward
 storage.py | 2 ++
Deleted branch feature/save-records (was d05376a).
* d05376a Add load_records stub
* 71b8a6e Print the week in the banner
* c4f930b Add project skeleton
```

Branch Names and Habits

A good branch name is a to-do item.

- feature/save-records, fix/negative-minutes.
- One feature per branch. Merge within days.
- git branch lists branches. * marks yours.
- Commit before you switch branches.

> **TIP** Check the branch name in the bottom-left corner of VS Code before you type.

SHELL

```
git branch
```

OUTPUT

```
  feature/report-title
* main
```

When Two Edits Collide

Same line, changed differently on two branches. Git will not guess.

- main changed TITLE to "Minutes per course".
- feature/report-title changed it to "Study time this week".
- Git stops and marks the file. UU means unmerged.

SHELL

```
git merge feature/report-title
git status --short
cat report.py
```

OUTPUT

```
CONFLICT (content): Merge conflict in report.py
Automatic merge failed; fix conflicts and then
commit the result.
UU report.py
<<<<<<< HEAD
TITLE = "Minutes per course"
=======
TITLE = "Study time this week"
>>>>>>> feature/report-title
```

Resolve in Three Moves

A conflict is a question. You answer it by editing the file.

- Edit: keep the line you want, delete all three marker lines.
- git add the file to say "answered".
- git commit to finish the merge.

> **TIP** VS Code offers Accept Current, Accept Incoming and Accept Both above each conflict.

SHELL

```
git add report.py
git commit -m "Merge report title: keep both ideas"
git log --oneline --graph
```

OUTPUT

```
*   f84e248 Merge report title: keep both ideas
|\
| * dd6fc23 Rename report title
* | b7d071f Clarify report title
|/
* 7846ed1 Add report title
```

Practice: Make a Conflict on Purpose

ACT 2 · BRANCH

Seven minutes, in a scratch folder. Not in your project.

- Make a repository with title.txt and commit it.
- On a branch, change the line and commit.
- On main, change the same line differently and commit.
- Merge, resolve, add, commit. Then git log --oneline --graph.

SHELL

```
mkdir conflict-practice && cd conflict-practice
git init
echo "Hello" > title.txt
git add title.txt && git commit -m "Add title"
git switch -c feature/greeting
echo "Hello, class" > title.txt
git commit -am "Greet the class"
git switch main
echo "Hi" > title.txt
git commit -am "Shorten the title"
git merge feature/greeting
```

BREAK

Halftime

Ten minutes. When you come back, open github.com and sign in.

- After the break: pull requests on your own repository.
- Then the lab: your repository, a feature branch and a merged pull request.

ACT 3 · PULL REQUEST

Pull Requests: Review Your Own Work

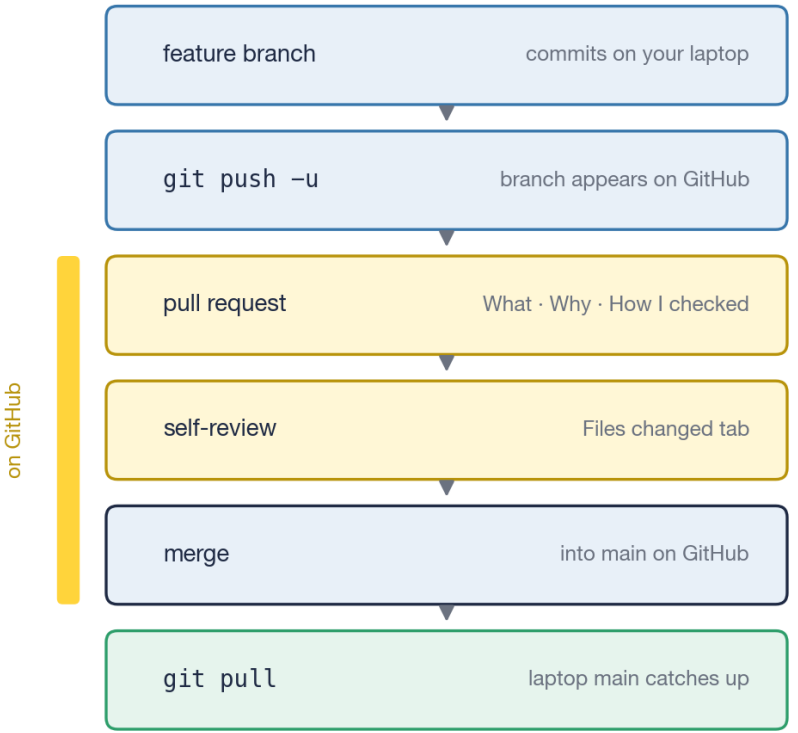
A pull request is a merge with a conversation attached.

- It lives on GitHub, between your branch and main.
- It is where you explain, check and get feedback.

Why a Pull Request in a Solo Project?

Nobody needs to approve it. It is still the best record of your work.

- The description says what changed and why, in your words.
- The Files changed tab forces you to read your own diff.
- The instructor can comment on a single line of your code.
- Each feature becomes one link: branch, commits, review, merge.



Push the Branch, Open the PR

ACT 3 · PULL REQUEST

This time we do not merge on the laptop.
We push the branch and let GitHub merge.

- base: main, compare: your feature branch.
- Title: what the feature does, like a commit message.
- Description: fill the four headings on the next slide.

SHELL

```
git switch -c feature/append-record
git add storage.py
git commit -m "Add append_record with header"
git push -u origin feature/append-record
```

OUTPUT

```
remote: Create a pull request for
remote: 'feature/append-record' on GitHub:
remote: https://github.com/jiwoo-park/
remote:   study-logger/pull/new/feature/append-record
 * [new branch]      feature/append-record
```

A PR Description That Earns Points

Four short headings. Two minutes to write, a whole feature explained.

- What: the change, linked to a feature ID.
- Why: the contract or criterion from your design document.
- How I checked: the command you ran and what it printed.
- Not in this PR: what is left for the next branch.

OUTPUT

```
## What
Add append_record() in storage.py (F1).

## Why
Contract: write the header only if the file is new.

## How I checked
Ran main.py twice -> one header, two rows.

## Not in this PR
Sorted totals come in feature/sort-totals.
```

The Self-Review Checklist

Read the Files changed tab as if a stranger wrote it.

- python3 main.py still runs from a fresh terminal.
- Every new function matches its contract and docstring.
- No leftover debug print() or commented-out experiments.
- No ignored files slipped in: __pycache__, .venv, .env.
- Only one feature in this pull request.

TIP Found a problem? Fix it on the same branch and push again. The PR updates itself.

Merge, Then Sync Your Laptop

ACT 3 · PULL REQUEST

Merge on GitHub. Your laptop does not know until you pull.

- On GitHub: Merge pull request, then Delete branch.
- On your laptop: back to main, pull, delete the local branch.
- Start the next feature branch from the updated main.

TRAP Skip git pull and your next branch starts from the old main.

SHELL

```
git switch main
git pull
git branch -d feature/append-record
```

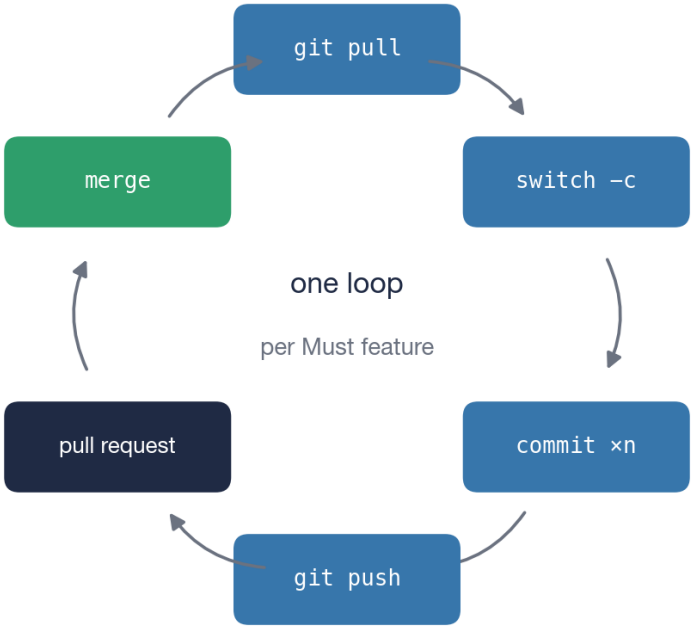
OUTPUT

```
Updating d05376a..e2a91c4
Fast-forward
 storage.py | 9 +++++++++
Deleted branch feature/append-record (was 5a0c7e1).
```

The Weekly Loop

One loop per Must feature. Weeks 11 to 13 are three or four loops.

- pull, branch, commit a few times, push, pull request, merge, pull.
- Small loops finish. Giant branches rot.
- The grader sees this rhythm in your history.



LAB

Lab: Your Repository, Your First PR

30 minutes. You leave with two links: a repository and a merged pull request.

- Starter: labs/session2/starter-repo/. Checklist: labs/session2/git-lab.md.
- Step 1: repository. Step 2: feature branch. Step 3: pull request.

Lab Step 1: The Repository

LAB

10 minutes. Copy the starter into a folder named after your project.

- Write your one-sentence goal at the top of README.md.
- Run python3 main.py once. It prints "Loaded 0 records".
- Create an empty GitHub repository with the same name.
- Commit, connect, push.

SHELL

```
cd my-project
git init
git add .
git commit -m "Add project skeleton"
git remote add origin \
  https://github.com/<you>/my-project.git
git push -u origin main
```

Lab Step 2: A Feature Branch

LAB

12 minutes. Turn your design document into stubs.

- git switch -c feature/skeleton.
- Rename the stub functions to the names in your design document.
- Copy each contract into its docstring. Bodies stay TODO.
- Commit at least twice, with real messages.
- python3 main.py must still run. Then push the branch.

PYTHON

```
# A stub: final name, full contract, no body yet
def make_record(date, course, minutes_text):
    """Return one record dict.

    Raises ValueError if course is empty
    or minutes is not positive.
    """
    raise NotImplementedError("make_record")
```

Lab Step 3: Pull Request and Merge

LAB

8 minutes. Open, review, merge, sync.

- Open a pull request from feature/skeleton into main.
- Fill What, Why, How I checked, Not in this PR.
- Walk the self-review checklist on the Files changed tab.
- Merge, delete the branch, then git switch main and git pull.

OUTPUT

```
$ git log --oneline --graph
*   3b8e0f1 Merge pull request #1 from
|\            you/feature/skeleton
| * 9c41d2a Copy contracts into docstrings
| * 52f7a90 Rename stubs to design names
|/
* 1a6c3e8 Add project skeleton
```

When Git Says No

LAB

Read the last line of the message. It usually names the fix.

- not a git repository: you are in the wrong folder. cd into the project.
- nothing to commit: you did not save the file, or forgot git add.
- src refspec main does not match any: make a first commit before pushing.
- Authentication failed: sign in through VS Code or gh auth login.
- Your branch is behind: run git pull, then continue.

> **TIP** git status answers most questions. Run it before and after every command.

Exit Check

Answer in one line each, then compare with a neighbour.

- What does git add do that git commit does not?
- Which four files or folders belong in .gitignore for this course?
- Why keep main working when nobody else uses your repository?
- What are the three moves that end a merge conflict?
- What must you run on your laptop after merging on GitHub?

Submit and Week 11

Your design has a home. Next week it starts to run.

- Submit via the LMS: repository URL, merged PR link and your git log --oneline --graph.
- Private repository? Add the instructor as a collaborator first.
- Must scope is now frozen. New ideas go to Should or Could.
- Week 11: implement the Must features, one branch and one pull request each.

> **RULE** Individual work. Every commit in your repository is yours.